Qahera at IslamicEval 2026: Corpus-Grounded Retrieval and Iterative Harness Search with a 4B-Parameter LLM for Islamic Citation Verification

Mohamed Abdelmagid
Team Qahera
mabdelmagid@aucegypt.edu

Hazem Ajlan
Team Qahera
hazemdiaa@aucegypt.edu

Abstract

We describe the Qahera system for the four IslamicEval 2026 subtasks: (1) detecting Qur'an/Hadith citation spans in LLM-generated Islamic answers, (2) judging citation authenticity, (3) correcting inauthentic citations against reference corpora, and (4) judging whether an authentic citation answers the question asked. Under a 13B-parameter open-model constraint, running entirely on free Kaggle GPUs, we combine a lightweight token-classification model with character-level auxiliary features (Subtask 1), corpus-grounded fuzzy retrieval (Subtasks 2–3), and a 4B-parameter instruction-tuned LLM (Gemma-3-4B) used only where retrieval alone cannot discriminate (Subtasks 3–4). Rather than hand-designing LLM prompts once, we develop them through an iterative *propose-evaluate-diagnose* search loop that reads full prediction traces after every candidate and hunts for systematic biases before each change. This surfaces small-model failure modes – a near-universal "reject" bias in isolated yes/no verification, and an "always name something" bias in numbered-list prompts – that would otherwise silently cap performance, and shows that validating candidates on small subsets can be actively misleading. Our final system reaches 0.9716 F1 (Subtask 1), 0.906 accuracy (Subtask 2), 0.773 accuracy (Subtask 3), and 0.673 macro-F1 (Subtask 4) on the official test sets.

1 Introduction

IslamicEval 2026 asks systems to audit Qur'anic and Hadith citations inside answers generated by large language models to real Islamic questions (Alharbi et al., 2026). A generated answer may quote a verse or a Hadith text (*matn*), attribute it to a source (*claimed_source*), and give its chain of narration (*isnad*); any of these can be fabricated, misquoted, or irrelevant to the question. The shared task decomposes this into four subtasks: **Subtask 1** (Span Detection) locates and types each citation segment in the raw response text; **Subtask 2** (Hallucination Identification) labels each Ayah/matn/isnad/claimed_source segment as `correct` or `incorrect`; **Subtask 3** (Correction) supplies the authentic canonical text for incorrect Ayah/matn segments, or the literal string خطأ ("error") when no faithful correction exists; and **Subtask 4** (Answer Relevance) judges whether an authentic citation directly or indirectly answers the question, since topical overlap alone is insufficient.

We operated under a constraint of open models with at most 13B parameters, on free-tier Kaggle GPUs (dual NVIDIA T4). This ruled out proprietary large-LLM judges and shaped our strategy: use cheap, well-understood string-level methods wherever the problem structure allows it, and reserve LLM calls for the sub-decisions where they are needed. For Subtask 4 and the harder half of Subtask 3 (Qur'anic correction), a naive, once-written LLM prompt underperforms a much simpler retrieval baseline, and the gap is explained by specific, reproducible biases of the small judge model rather than prompt wording alone. We therefore developed these components through an iterative search loop (§4) that treats every candidate's full per-example trace as evidence for diagnosing *why* it failed, not just how well it scored.

2 Data and Resources

The training set contains 4,706 model responses (over 15,000 annotated citations); we held out 484 as a question-disjoint dev split for Subtask 1, and used the organizers' `dev_set` files for Subtasks 2–4. Two reference corpora are provided: 6,236 Qur'anic verses and 34,994 Hadith entries from six canonical collections (Bukhari, Muslim, Abu Dawud, Tirmidhi, Nasa'i, Ibn Majah); Subtasks 2 and 3 require every prediction to be grounded in these corpora.

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1088–1092
October 28-29, 2026 ©2026 Association for Computational Linguistics

3 System Description

Figure 1 shows the finalized pipeline for each subtask.

3.1 Subtask 1: Span Detection

We frame span detection as character-level token classification into five labels (`neither`, `Ayah`, `matn`, `isnad`, `claimed_source`), decoded into spans by taking maximal same-label runs (Figure 1a). The backbone is AraBERTv02-base (Antoun et al., 2020); its hidden states are concatenated with a linear projection of five character-level auxiliary features: local tashkeel (diacritic) density, an inside-quotation-marks flag, and three gazetteer hit indicators – 114 surah names, six Hadith-collection names, and common isnad markers such as حدثنا ("he narrated to us"). Class weights compensate for the heavy `neither` majority class, and a 512-token, 128-stride sliding window sums logits across overlaps at inference. Regex post-hoc correction rules for `claimed_source` spans were tested but rejected: they lowered validation F1 (0.968 vs. 0.971).

3.2 Subtask 2: Hallucination Identification

We do not train a classifier for this task; instead we deterministically check each segment against the two provided corpora, labeling it `correct` if located there and `incorrect` otherwise (Figure 1b). Before matching we normalize the Arabic text: strip diacritics and elongation marks, and unify inconsistently written letter shapes (alef/hamza forms, final ة vs. ه, and ى vs. ي).

Ayah and matn. We put all 6,236 Qur'anic verses into one long string and all 34,994 Hadiths into another, and look for a segment's normalized text inside the matching string; an exact hit is `correct`. We deliberately use exact matching – fabricated verses usually look very close to real ones, so a loose fuzzy match would let them through – but exact matching alone is too strict, since a real quotation can have small spelling differences from the canonical text. When the exact match fails, we try one fuzzy step: a partial-match score (`rapidfuzz.partial_ratio`) of at least 94 for Ayah or 92 for matn is still accepted as `correct`. These thresholds work because the two groups sit far apart: real verses that miss the exact match still score 95–100, while altered ones score 60–85. For matn, we first use a word index to narrow candidate Hadiths before running the partial match, given the large corpus.

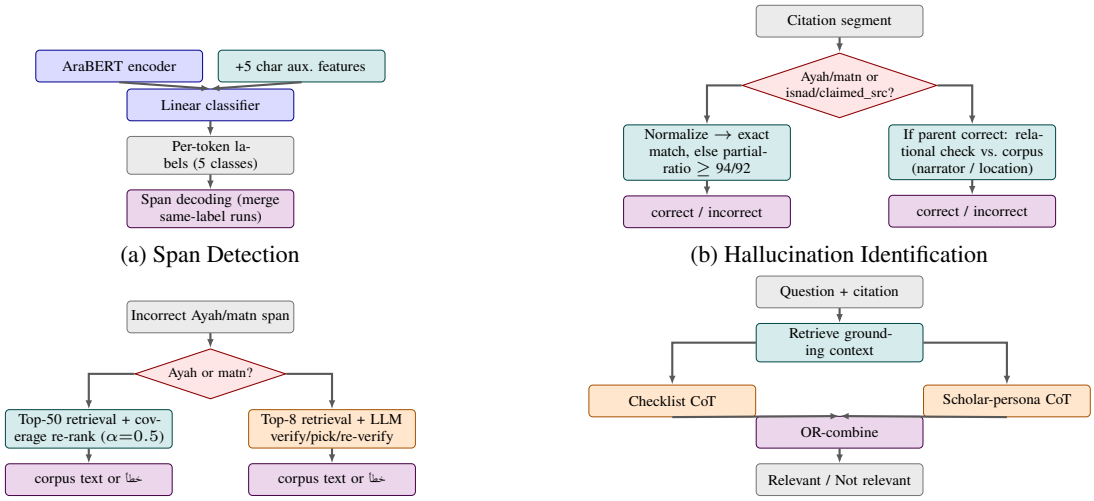
Isnad. A chain of narration cannot be judged from its own text: correct and incorrect chains often look almost identical (e.g., the same companion, عن أبي هريرة, "on the authority of Abu Hurairah," appears in both). What matters is whether the corpus attributes *this* Hadith text to *this* narrator: we locate the segment's parent matn in the corpus, read the narrator's name from the isnad, and check whether that name co-occurs with the matn there. If so, the isnad is `correct`; if the matn appears only with other narrators, it is `incorrect`. (The isnad dev set is only 30 segments, so this number is noisy.)

Claimed_Source. We again check the claim against the corpus, not in isolation. For a Qur'anic source, we read the surah name and verse number from the claim, look up the real verse there, and compare it with the parent Ayah using a partial-match ratio (threshold 75); a match is `correct`, a valid location with a different verse is `incorrect`. For a Hadith source, we read the claimed collection (e.g., Bukhari) and check whether the parent matn appears there, using one text index per book. Per the subtask's rules, `claimed_source` and `isnad` are scored only when their parent segment is itself correct.

On the development set, this pipeline scores 0.884 macro accuracy: 0.961 for Ayah, 0.942 for matn, 0.733 for isnad, and 0.900 for `claimed_source`. Every label comes from looking something up in the corpus; exact matching catches fabrications, and a small amount of fuzzy tolerance keeps real citations from being rejected over minor spelling differences.

3.3 Subtask 3: Correction

Correction solves two different problems depending on segment type (Figure 1c), treated differently for reasons grounded in evidence, not assumption. For **Ayah**, plain `token_set_ratio` is bag-of-words: a short *generic* shared phrase (e.g. على صراط مستقيم, "on a straight path," a formula recurring across many unrelated verses) can score 100 against a totally unrelated verse, whereas a genuine (possibly truncated) quotation shares most of itself *contiguously* with the true verse. We retrieve a large candidate pool (top-50) and re-rank it by a combined score $\alpha \cdot \text{token_set_ratio} + (1-\alpha) \cdot \text{coverage}$, where *coverage* is the longest contiguous substring shared between query and candidate, as a fraction of the query's own length. The top candidate's combined score is thresholded (α=0.5, cutoff 50 – both from a



(a) Span Detection

(b) Hallucination Identification

(c) Correction

(d) Answer Relevance

Figure 1: Finalized pipelines for all four subtasks. Gray = input/output, blue = neural encoder, teal = corpus retrieval, orange = LLM call, violet = decision/merge, red diamond = branching question.

stable plateau in a grid sweep rather than the single best point, to avoid over-fitting); below threshold we emit خطأ. No LLM call is used for Ayah correction: an isolated yes/no verifier showed a near-universal reject bias on genuine paraphrases (§6).

For **matn**, retrieval alone cannot disambiguate near-duplicate Hadiths sharing only generic isnad-adjacent boilerplate (e.g. رسول الله صلى الله عليه وسلم, "the Messenger of Allah, peace be upon him"), so we add a three-stage Gemma-3-4B-it (Gemma Team, Google DeepMind, 2025) pipeline over the top-8 retrieved candidates: (i) an isolated yes/no call asks whether the fuzzy top-1 candidate is genuinely the same Hadith as the (possibly corrupted) span; (ii) if rejected, a second call presents the remaining candidates as a numbered list and asks the model to name the correct one, or 0 for none; and (iii) whatever is named in step (ii) is re-verified with the *same* isolated yes/no framing as step (i) before acceptance. Step (iii) exists because step (ii)'s list-choice framing has a measurable bias toward naming *some* candidate even when none is correct (§6); re-checking with the more reliable isolated framing recovers much of the lost precision. The model never re-generates citation text – only retrieved, corpus-verbatim text is returned – since Qur'anic scoring is diacritic-sensitive and free generation risks subtly wrong diacritics.

3.4 Subtask 4: Answer Relevance

For each citation (Figure 1d), we retrieve its closest corpus match (grounding context only, not a decision signal) and query Gemma-3-4B-it with two independently-worded chain-of-thought prompts: a *checklist* framing walking through the task's direct / indirect-case-1 / indirect-case-2 / topical-overlap-trap distinctions, and a *scholar-persona* framing asking the model to reason as a mufti would about whether the citation is legitimate evidence for this question. The citation is labelled relevant if *either* call says so. An earlier three-way design added a *devil's-advocate* framing (argue the citation is irrelevant, then check if the argument holds); back-testing its stored votes showed it answered relevant on 76% of gold-non-relevant examples regardless of content – it always talked itself into "relevant" – with no discriminative signal beyond the other two, so we dropped it, cutting average LLM calls per example from 3 to roughly 2 at unchanged accuracy.

4 Iterative Harness Search

For Subtasks 3–4, we did not hand-tune a single LLM pipeline, but used an iterative loop – in the spirit of end-to-end harness optimization (Lee et al., 2026), organized around *qualitative* trace diagnosis rather than a purely numeric objective: (1) evaluate the current harness on the full development set and log every example's inputs, calls, and outputs; (2) inspect the wrong-prediction traces for a systematic pattern; (3) propose one concrete, evidence-justified change; (4) re-evaluate, and revert if it does not hold up. Figure 2 summarizes this cycle, which surfaced three findings about small-model failure modes, discussed in §6.

5 Results

Table 1 reports official Codabench test-set scores. All development-set numbers use the organizers'

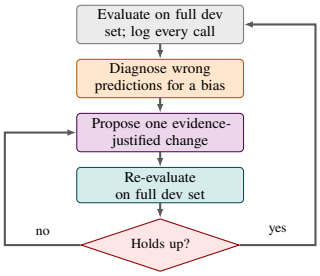


Figure 2: The propose-evaluate-diagnose loop used to develop the Subtask 3–4 harnesses (see §6 for why full-set scoring matters).

Subtask	Metric	Dev	Test
1: Span Detection	F1	0.971	0.972
2: Hallucination ID	Acc.	0.884	0.906
3: Correction	Acc.	0.763	0.773
4: Answer Relevance	Macro-F1	0.706	0.673

Table 1: Official development and test results. Subtask 2's development score is the macro accuracy over the four segment types on the full dev split.

full `dev_set` files, never a subset.

Subtask 2's test breakdown mirrors development: 0.945 Ayah, 0.889 matn, 0.852 isnad, and 0.937 claimed_source, close to dev (0.961 / 0.942 / 0.733 / 0.900) despite the isnad slice's small size. For Subtask 3, Ayah accuracy is 0.715 and matn is 0.824 on test, closely tracking dev (0.710 / 0.816). For Subtask 4, dropping from 0.706 (dev) to 0.673 (test) macro-F1 is a smaller, more typical gap than the subset-vs-full-dev gaps in §6, suggesting the two-vote ensemble is not overfit to dev.

Non-LLM baselines surpassed during development: 0.566 accuracy for Subtask 3 (fixed-cutoff retrieval, no re-ranking or LLM verification) and 0.573 macro-F1 for Subtask 4 (a fine-tuned AraBERT cross-encoder), both on the full dev set.

6 Discussion

The iterative harness search described in §4 surfaced three findings about small-model behavior that we consider more broadly useful than any single prompt:

Small-subset validation is unreliable. A 60-example subset suggested 0.694 Ayah correction accuracy for the retrieval-only baseline; the coverage-rerank policy we eventually adopted looked *worse* on that subset (0.639), arguing against shipping it. On the full 943-row development set the opposite was true: coverage-rerank reached 0.710, a clear improvement over 0.641. We validated every candidate on the full split before trusting it.

Isolated binary verification has a strong reject bias for Qur'anic paraphrase. Adding an LLM yes/no check asking whether a retrieved verse was the one quoted collapsed recall on genuinely correct Ayah matches from 89/219 to 23/219: the raw outputs showed "no" to matches that were, in 66 of 100 inspected cases, an exact substring or extension of the correct verse – precisely what its own prompt permitted. The bias did not respond to more explicit wording, so we removed the LLM check for Ayah entirely.

Numbered-list choice prompts have an "always answer something" bias. Across three iterations, whenever the matn pipeline showed the model a numbered candidate list and asked it to name the correct one or answer 0, it named a concrete (often wrong) candidate roughly three times as often as 0, even when none was genuinely correct. The fix was structural: re-verify whatever the list-choice step names using the isolated single-candidate yes/no framing, which does not exhibit this bias.

Together, these findings share one lesson: a small LLM's failure modes are structural, not random, and can be routed around once diagnosed. This is why LLM-based verification underperformed algorithmic re-ranking for Ayah: the model's judgment carried a stronger bias than the retrieval scorer. It was worth its cost for matn and Subtask 4, where judgment requires weighing semantic content – but even there, the working designs succeed by containing a known bias structurally, not trusting a single call. Under a small-model budget, diagnosing failure beats prompting harder.

Limitations

Our Subtask 3 retrieval index uses only surface fuzzy matching; roughly 7% of genuinely correctable citations in our full-dev audit never appeared among retrieved candidates, a recall ceiling a semantic (embedding-based) index might close. The biases we found are not guaranteed exhaustive; a different held-out split could surface others, and our harness search ran for few iterations (nine for Subtask 3, four for Subtask 4). Finally, all LLM components use a single 4B-parameter model (Gemma-3-4B-it); we cannot say how much of the observed bias is specific to it versus small instruction-tuned LLMs generally.

Acknowledgments

We thank the IslamicEval 2026 organizers for the dataset, corpora, and infrastructure.

References

Rahaf Alharbi, Abdulelah Alturki, Watheq Mansour, Rana Malhas, Hamdy Mubarak, Kareem Darwish, Tamer Elsayed, and Walid Magdy. 2026. IslamicEval 2026: The second shared task of capturing LLMs hallucination in Islamic content. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France. European Language Resource Association.

Gemma Team, Google DeepMind. 2025. Gemma 3 technical report. Google DeepMind. Model card and technical report for the Gemma 3 family of open models.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *Preprint*, arXiv:2603.28052.